

BÁO CÁO BÀI TẬP TUẦN 4

Mã hóa Sequential Counter cho ràng buộc thứ tự (B3)

1 Mục tiêu và phương pháp

Tuần 4 tập trung vào ràng buộc **Precedence** (thứ tự đi đường của một tàu trên hai ga liên tiếp $r \prec_i q$), tạm gọi là B3: nếu tàu i xuất phát ở ga r tại thời điểm h_{ir}^p thì nó chỉ được xuất phát ở ga q sau $h_{ir}^p + l_r$ (với l_r là thời gian chiếm dụng tối thiểu ga r). Theo hướng dẫn, mục tiêu là xây dựng mã hóa **Sequential Counter** trên biến tích lũy $R_{iq,1,t}$.

Các file trong thư mục `submissions/tuan4_doquangvinh/`:

- `bai1_scl_precedence.py` - mã hóa SCL (C1-C7), mã hóa Direct baseline và ràng buộc Exactly-One;
- `bai2_bruteforce_checker.py` - bộ duyệt vét cạn kiểm chứng tính tương đương nghiệm giữa Direct và Sequential Counter;
- `run_experiments_week4.py` - benchmark 12 cấu hình và vẽ 3 biểu đồ chỉ tiêu;
- `experiment_results.csv`, `images/*.png` - số liệu và trực quan hóa.

2 Mã hóa Sequential Counter for Ladder cho ràng buộc thứ tự (Bài 1)

Ý tưởng cốt lõi: thay vì sinh mệnh đề cho từng cặp interval (x_{ir}^p, x_{iq}^t) như Direct (bùng nổ $O(n \times m)$), ta đưa vào m biến tích lũy $R_{iq,1,t}$ với ý nghĩa

$$R_{iq,1,t} = 1 \Leftrightarrow \text{tàu đã xuất phát ở ga } q \text{ tại interval } \leq t.$$

Nhờ đó mỗi interval p của ga r chỉ cần 1 mệnh đề: nếu r xuất phát tại thời điểm h_{ir}^p thì q phải bắt đầu sau $h_{ir}^p + l_r$, tức $\neg R_{K_p}$ với $K_p = \max\{t \mid h_{iq}^t < h_{ir}^p + l_r\}$. Bộ mệnh đề đầy đủ (C1-C7):

- (C1) $R_1 \rightarrow x_1^q$ và (C2) $x_1^q \rightarrow R_1$ - base case;
- với $t = 2 \dots m$: (C2) $x_{iq}^t \rightarrow R_{iq,1,t}$; (C3) $R_{iq,1,t-1} \rightarrow R_{iq,1,t}$ (tính đơn điệu của biến tích lũy); (C4) $R_{iq,1,t} \rightarrow R_{iq,1,t-1} \vee x_{iq}^t$; (C5) $R_{iq,1,t-1} \rightarrow \neg x_{iq}^t$;
- (C6) $R_{iq,1,m}$ (đơn nguyên - ép tàu chắc chắn phải xuất phát ở ga q);
- (C7) $x_{ir}^p \rightarrow \neg R_{iq,1,K_p}$ (rút gọn biên: nếu $K_p = 0$ bỏ qua, $K_p = m$ cấm hẳn x_p^r).

Số mệnh đề sinh ra đúng bằng $4m - 1 + n$ và số biến $= (n + m) + m$ (thêm m biến R), tức **tuyến tính** $O(n + m)$, so với $O(n \times m)$ của mã hóa trực tiếp vì Direct phải xét mọi cặp quá sớm.

3 Kiểm chứng tính tương đương nghiệm bằng brute-force (Bài 2)

Script `bai2_bruteforce_checker.py` kiểm chứng: công thức CNF ($SCL + Exactly-One$) có diễn đạt đúng ràng buộc vật lý không:

- (i) `physical_b3_ok` - định nghĩa nghiệm vật lý: $Exactly-One$ trên $x_{iq} + precedence\ B3$;
- (ii) `cnf_satisfied` với $Direct + Exactly-One$;
- (iii) `cnf_satisfied` với $SCL + Exactly-One$ khi có gán biến R .

Với mỗi gán biến quyết định (duyệt đủ 2^{n+m} tổ hợp bằng bitmask), chương trình xét thêm 2^m tổ hợp cho biến phụ R . Mismatch chỉ xảy ra khi kết quả của hai mã hóa CNF khác với kết quả vật lý. Chạy ở bốn giá trị biên $l_r \in \{0, 5, 15, 100\}$ (góc 0, khoảng giữa, biên, lớn hơn mọi interval), mọi phép gán đều cho **0 mismatch**. Kết luận: cả $Direct$ lẫn SCL đều tương đương nghiệm 100% với $B3 + Exactly-One$.

Hai điểm phát hiện trong quá trình kiểm thử đã được khắc phục trước khi hoàn thành:

- Phải cộng **Exactly-One** vào cả $Direct$ lẫn SCL để cả hai thao tác trên cùng tiền đề $B1$; nếu chỉ SCL có EO , các gán vi phạm $B1$ sẽ khiến $Direct$ “chấp nhận oan” và gây mismatch giả;
- SCL cần thêm chiều ngược của $(C1)$, tức $R_1 \rightarrow x_1^q$, để R_1 không bao giờ nhận giá trị 1 khi tàu chưa xuất phát - bảo toàn nghĩa tích lũy.

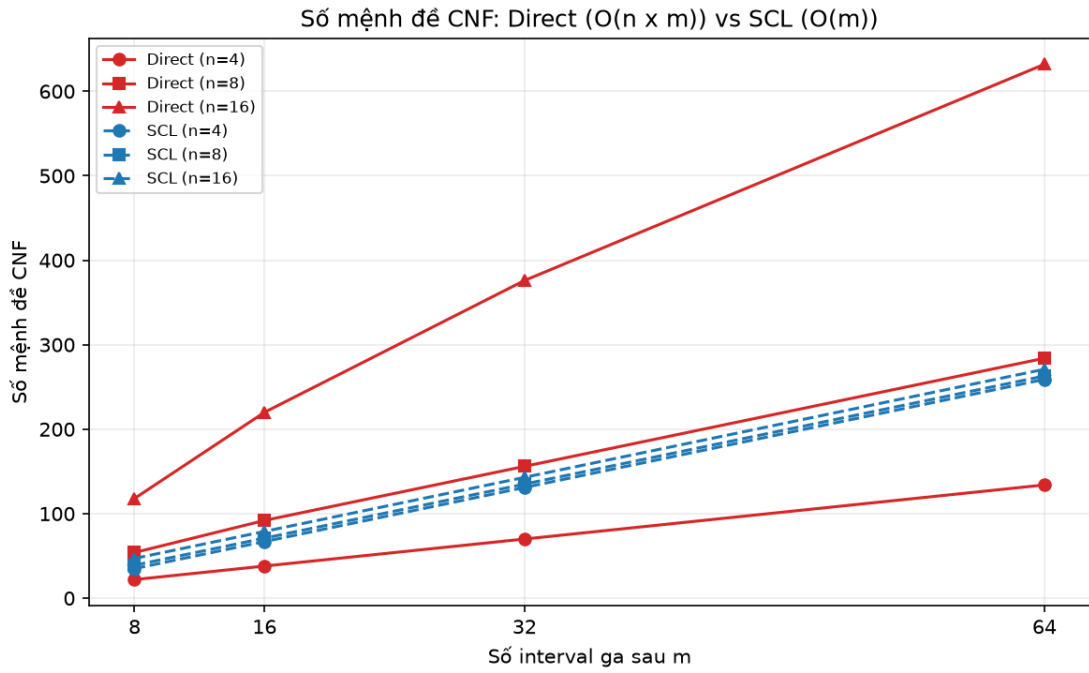
4 Kết quả thực nghiệm

Dùng `run_experiments_week4.py` quét 12 cấu hình với $m \in \{8, 16, 32, 64\}$, $n \in \{4, 8, 16\}$ và đo chỉ tiêu bằng solver `CaDiCaL153` qua `PySAT` (số liệu đầy đủ trong `experiment_results.csv`). Do chọn $l_r = 5 \times m$, mỗi interval p ở ga r chặn khoảng $\frac{m}{2}$ interval ở ga q : $Direct$ phải sinh một mệnh đề cho từng cặp trong khối đó (bùng nổ bậc hai), còn SCL chỉ tốn một mệnh đề mỗi p cộng chuỗi tuyến tính R .

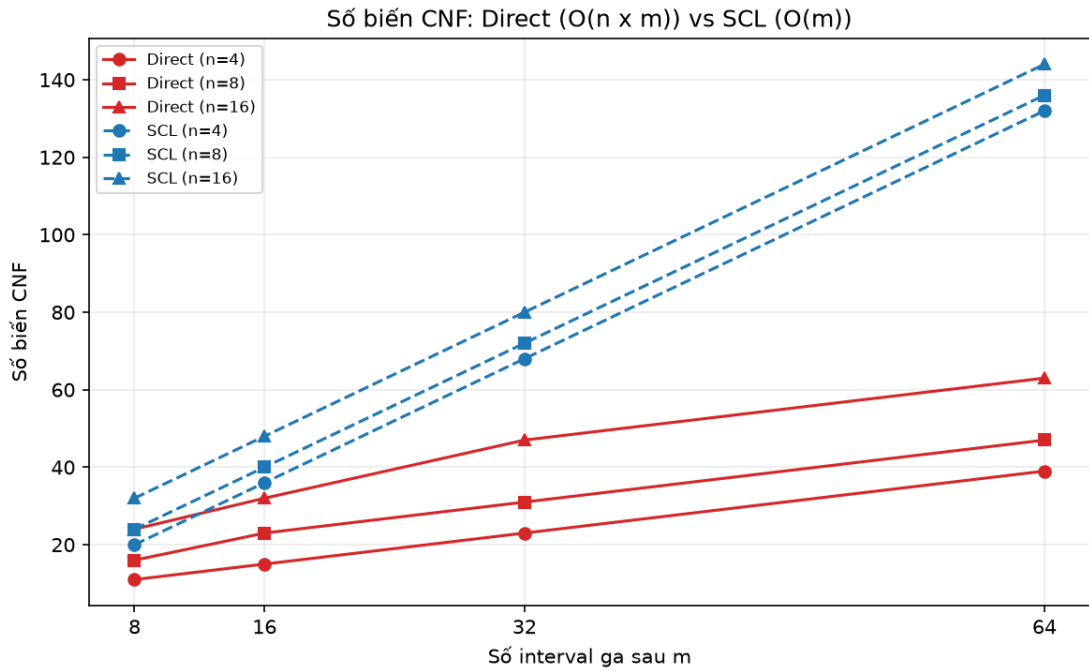
Bảng dưới tóm tắt số mệnh đề và số biến theo m (với $n = 16$ cố định):

Bảng 1: Số mệnh đề và số biến khi $n = 16$, $l_r = 5 \cdot m$.				
m	$Direct$ (clauses)	SCL (clauses)	$Direct$ (vars)	SCL (vars)
8	118	47	24	32
16	220	79	32	48
32	376	143	47	80
64	632	271	63	144

Hình 1 và 2 cho thấy xu hướng rõ rệt: khi m tăng, số mệnh đề $Direct$ tăng nhanh (bậc hai theo $n \times m$), còn SCL chỉ tăng *tuyến tính* theo $4m$ và gần như *không đổi* theo n . Đây chính là lợi thế quyết định của SCL khi số interval m và số ga n lớn.



Hình 1: Số mệnh đề: Direct $O(n \times m)$ vs SCL $O(n + m)$.



Hình 2: Số biến: SCL đổi thêm m biến R để giảm số mệnh đề.

5 Kết luận

Tuần 4 đã mã hóa ràng buộc thứ tự B3 bằng SCL (C1-C7) với biến tích lũy R , đạt số mệnh đề tuyến tính $4m - 1 + n$ thay vì bậc hai như Direct. Bộ brute-force xác nhận công thức tương đương 100% với ràng buộc vật lý B3 + Exactly-One trên toàn bộ không gian gán (duyệt 2^{n+m} gán $\times$ 2^m gán R). Thực nghiệm với CaDiCaL153 khẳng định SCL tăng tuyến tính trong khi Direct tăng bậc hai, chứng tỏ phương pháp đáng dùng khi m và n lớn.

6 Sản phẩm

- Bài 1: `bai1_scl_precedence.py` - mã hóa SCL (C1-C7), Direct, Exactly-One.
- Bài 2: `bai2_bruteforce_checker.py` - kiểm chứng tương quy logic.
- Bài 3: `run_experiments_week4.py` + `experiment_results.csv` + 3 biểu đồ trong `images/`.
- Báo cáo: `bao_cao_tuan_4.tex`.
- Repository Git: https://github.com/Viing3107/maxsattrainscheduling/tree/tuan4_doquangvinh